

作业 06

汪琦 2023010905015

有 N 个生产者进程和 M 个消费者进程共享一个大小为 K 的缓冲区，要求：

- 进程之间必须互斥访问缓冲区；
- 每一条放入缓冲区的数据，所有消费者都必须接收一次；
- 缓冲区满时，生产者必须阻塞；
- 缓冲区空时，消费者必须阻塞。

由于每条数据都要被所有消费者各读取一次，所以不能在某个消费者读取后立即释放该缓冲区单元。只有当 M 个消费者都读取过该数据后，该缓冲区单元才可以重新作为空位供生产者写入。因此，为每个消费者设置一个独立的读指针，并为缓冲区中的每个单元设置一个剩余读取次数 `remain[]`。

```
semaphore mutex = 1;
```

`mutex` 用于保证对缓冲区、读写指针以及 `remain[]` 数组的互斥访问。

```
semaphore empty = K;
```

`empty` 表示当前可供生产者写入的空缓冲区单元数量，初值为 K 。当缓冲区满时，生产者执行 `P(empty)` 会被阻塞。

```
semaphore full[M] = {0};
```

`full[i]` 表示消费者 i 当前可以读取的数据条数，初值为 0 。当消费者 i 没有可读数据时，执行 `P(full[i])` 会被阻塞。

共享变量：

```
#define K 缓冲区大小
```

```
#define M 消费者数量
```

```
Data buffer[K];           // 环形缓冲区
int in = 0;                // 生产者写指针
int out[M] = {0};          // 每个消费者各自的读指针
int remain[K];             // remain[j] 表示 buffer[j] 还需要被多少个消费者读取
```

```
semaphore mutex = 1;       // 互斥信号量
semaphore empty = K;       // 空缓冲区单元数量
semaphore full[M] = {0};   // 每个消费者对应的可读数据数量
```

生产者进程：

```
void producer() {
    while (true) {
        Data item = produce();
        P(empty); // 缓冲区满时阻塞
        P(mutex); // 互斥访问缓冲区
        buffer[in] = item;
        remain[in] = M; // 新数据需要被 M 个消费者各读取一次
    }
}
```

```

    in = (in + 1) % K;
    V(mutex);
    for (int i = 0; i < M; i++) {
        V(full[i]); // 通知每个消费者都有一条新数据可读
    }
}
}

```

消费者 i 的同步过程如下:

```

void consumer(int i) {
    while (true) {
        P(full[i]); // 对消费者 i 来说, 缓冲区空时阻塞
        P(mutex); // 互斥访问缓冲区
        Data item = buffer[out[i]];
        remain[out[i]]--; // 当前消费者已读取该数据
        if (remain[out[i]] == 0) {
            V(empty); // 所有消费者均已读取, 释放该缓冲区单元
        }
        out[i] = (out[i] + 1) % K;
        V(mutex);
        consume(item);
    }
}
}

```